POLITECHNIKA BYDGOSKA

Informatyka Stosowana

III rok studiów | V semestr | grupa 1

Programowanie w Środowisku Windows

Rok akademicki 2026/2027

Qraft

Specyfikacja i projekt systemu kontroli jakości regranulatu PP

C#/.NET · ASP.NET Core · Vue · RabbitMQ · SQLite

Mateusz Dalke, nr indeksu 124382
Oskar Kirko, nr indeksu 123758

Bydgoszcz 2026

1. Cel, użytkownik i granica zakresu

Qraft wspiera ocenę jakości regranulatu polipropylenowego PP. Pracownik jakości potrzebuje czytelnego wyniku trzech próbek, wskazania niespełnionych kryteriów i zachowania danych źródłowych. Symulator zastępuje źródło pomiarów i pozwala powtarzać przypadki poprawne oraz błędne bez fizycznych urządzeń.

Obecny zakres kodu to pionowy przepływ: symulator → RabbitMQ → walidacja i ocena Qraft → trwały wynik → RabbitMQ → zapis odpowiedzi w symulatorze. Panel Vue prezentuje dane obu usług. Ten przepływ należy odróżnić od docelowego procesu ręcznych decyzji produkcyjnych.

C#/.NET 10 realizuje logikę, API i zadania w tle. ASP.NET Core udostępnia interfejsy HTTP, Vue 3 z TypeScript stanowi aktualny interfejs developmentu, RabbitMQ przekazuje komunikaty, a dwie niezależne bazy SQLite przechowują dane Qraft i symulatora.

Pierwotny projekt zakładał interfejs Windows/WPF. Obecny kod wykorzystuje interfejs webowy rozwijany na macOS. Zgodność tej formy interfejsu z wymaganiem aplikacji Windows wymaga ustalenia z prowadzącym; dokument nie stwierdza jego akceptacji.

Poza zakresem obecnej implementacji pozostają: pełny model partii, ręczne decyzje E1/E2 i wymuszanie ich kolejności, przygotowanie materiału, poprawa, zwolnienie i odrzucenie partii oraz raport całego cyklu. Zarządzanie magazynem i produkcją nie jest celem systemu.

Tabela 1. Status funkcji

Obszar	**Status**
Walidacja, ocena, komunikaty i trwałość	Zaimplementowany przepływ odbioru, oceny i odpowiedzi; wyniki sprawdzenia w rozdziale 8.
Panel scenariuszy i wyników	Bieżący interfejs Vue; bez ręcznej obsługi cyklu partii.
Decyzje, poprawa, zwalnianie partii	Planowany zakres domenowy.
Zaliczenie z interfejsem Vue	Akceptacja niepotwierdzona.

2. Reguły oceny i scenariusze

Badanie wymaga dokładnie trzech próbek o niepowtarzających się kodach. Każda zawiera wilgotność, gęstość materiału i logiczną ocenę zgodności barwy. Próbka jest zgodna wyłącznie wtedy, gdy spełnia wszystkie trzy kryteria. Badanie jest zgodne, gdy zgodne są wszystkie próbki.

Limity w tabeli 2 są danymi dydaktycznymi, a nie normami przemysłowymi. Gęstość oznacza gęstość materiału, nie gęstość nasypową. Liczba niezgodności zlicza niespełnione kryteria próbek; nie jest procentem wadliwej masy.

Przykład: wilgotność 0,280% przy maksimum 0,200% jest poprawną wartością pomiarową, ale daje niezgodność jakości. Brak wymaganej wilgotności jest błędem danych i prowadzi do odrzucenia komunikatu. Wynik jakości nie jest decyzją o zwolnieniu materiału.

Tabela 2. Kryteria oraz pięć scenariuszy

Kryterium / scenariusz	**Reguła / oczekiwany rezultat**
Wilgotność	0,000–0,200% włącznie.
Gęstość	880–950 kg/m³ włącznie.
Barwa	Zgodna ze wzorcem: true.
good	Processed / Conformant.
nonconformant	Processed / Nonconformant; wskazane niespełnione kryteria.
invalid-payload	Rejected; błędy wymaganych danych.
malformed-json	Rejected; błąd odczytu JSON.
boundary	Processed / Conformant dla wartości na granicach.

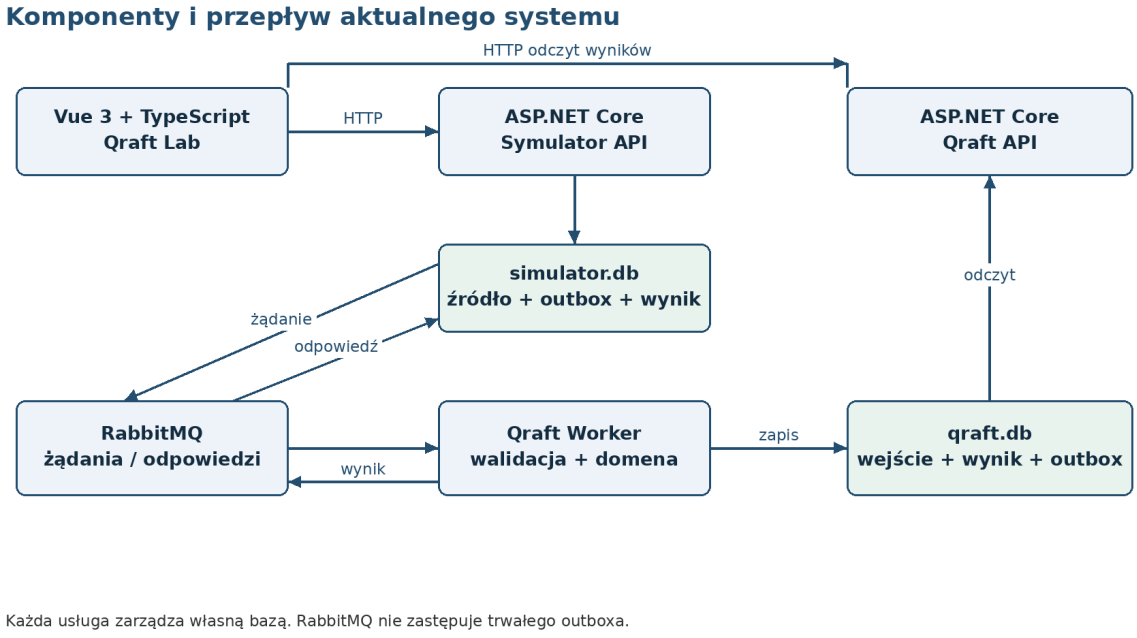
3. Architektura i przepływ komunikatów

Symulator przechowuje scenariusze i tworzy przebieg z komunikatami źródłowymi oraz zadaniami wysyłki. Zapis tych elementów jest wykonywany przed publikacją. Worker Qraft odbiera wiadomość, waliduje kontrakt, wywołuje domenę i zapisuje wynik z outboxem odpowiedzi.

Odpowiedź wraca przez osobną kolejkę. Symulator sprawdza identyfikatory oraz spójność treści i zapisuje wynik przy źródle. Qraft API i API symulatora służą do odczytu i sterowania lokalnym panelem; interfejs nie zapisuje bezpośrednio w cudzej bazie.

Domain zawiera reguły i obiekty wartości bez zależności od EF Core oraz RabbitMQ. Application realizuje walidację i procesor, korzystając z IQualityStore. Infrastructure implementuje zapis SQLite. Contracts definiuje DTO i nazwy kolejek, Messaging obsługuje transport. Worker łączy przetwarzanie i publikację outboxa.

Interfejs IQualityStore oddziela przypadek użycia od magazynu danych. Rekordy C# opisują kontrakty i wyniki, LINQ agreguje oceny, a Task i CancellationToken obsługują asynchroniczne I/O i zatrzymanie usług. Zastosowane abstrakcje wynikają z granic odpowiedzialności.



Rysunek 1. Obecny podział komponentów i przepływ danych. Źródło: opracowanie własne.

4. Model danych i trwały magazyn

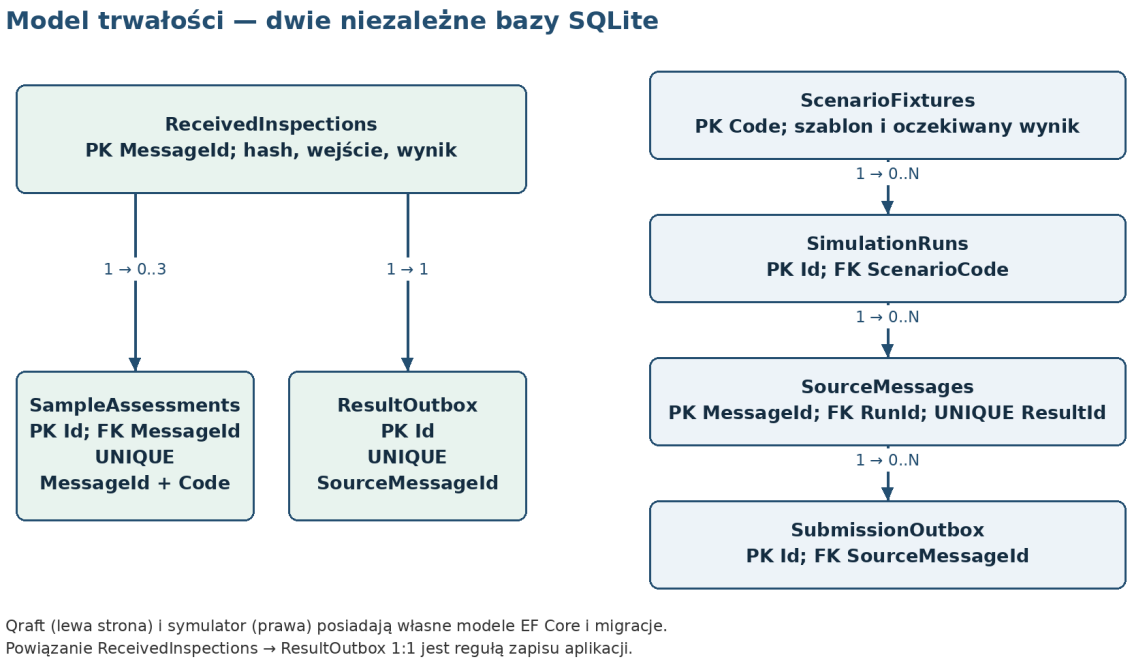
Przepływ wykorzystuje dwie niezależne bazy: qraft.db i simulator.db. SQLite odpowiada lokalnemu stanowisku demonstracyjnemu; nie wymaga oddzielnego serwera bazy. Obie usługi mają własny model EF Core i migracje wykonywane przy starcie. Nie należy usuwać baz ani wolumenu brokera w celu obejścia błędu.

Qraft przechowuje odebrane wejście, hash SHA-256 i pełny wynik w ReceivedInspections. Kluczem jest MessageId. SampleAssessments zawiera ocenę i kryteria próbek, z unikalną parą MessageId/Code. ResultOutbox przechowuje odpowiedź i stan publikacji, z unikalnym SourceMessageId.

Jeden odebrany komunikat ma zero do trzech zapisanych ocen próbek: dla odrzucenia oceny nie powstają. Odpowiedź jest utrwalana wraz z wejściem. Każda próbka ma dokładnie jeden rekord nadrzędny. Outbox ma własny identyfikator i odniesienie do źródła; unikalność SourceMessageId zapobiega podwójnemu zadaniu wyniku. Relacja wejścia z outboxem 1:1 na rysunku 2 jest regułą zapisu aplikacji: unikalny indeks uniemożliwia drugi outbox tego samego źródła, lecz model EF nie wymusza kluczem obcym jego istnienia.

Symulator przechowuje cztery tabele: ScenarioFixtures (klucz Code), SimulationRuns (Id, klucz obcy ScenarioCode), SourceMessages (MessageId, klucz obcy RunId) i SubmissionOutbox (Id, klucz obcy SourceMessageId). Scenariusz ma wiele przebiegów, przebieg wiele komunikatów, a komunikat wiele zadań wysyłki, m.in. po replay. Odpowiedź jest zapisywana w SourceMessages razem z ResultId; unikalny indeks ResultId zapobiega przypisaniu tego samego wyniku do wielu źródeł. Klucze obce blokują usunięcie rekordu mającego zależne dane.

Zapis danych pomiarowych i wyników zachowuje historię. Retransmisja nie edytuje poprzedniej oceny. Usuwanie zatwierdzonych wyników i dowolna edycja danych nie są udostępnionymi operacjami biznesowymi. Endpoint /api/batches grupuje importowane wyniki; nie stanowi pełnej encji Partia.



Rysunek 2. Model trwałości Qraft i symulatora. Źródło: opracowanie własne.

5. Kontrakty, walidacja i błędy

InspectionSubmittedV1 zawiera schemaVersion = 1, messageId, simulationId, batchId, batchCode, productName, massKg, productionDate, stage, attempt, author, occurredAt i samples. Próbka zawiera code, collectedAt, moisturePercent, densityKgM3 i colorConformant. Stage przyjmuje E1 albo E2; sama etykieta nie wymusza kolejności procesu.

Walidacja obejmuje składnię JSON, obsługiwaną wersję schematu, identyfikatory, wymagane wartości, trzy próbki i poprawność danych. Deserializacja rozróżnia wielkość liter i odrzuca nieznane pola. Warstwa aplikacji oddziela błąd kontraktu od wyniku oceny jakości.

InspectionProcessedV1 zawiera ResultId, identyfikatory źródła i symulacji, status, wynik jakości, listę errors, oceny samples i czas przetworzenia. Processed / Conformant oznacza dane zgodne; Processed / Nonconformant oznacza poprawne dane o niezgodnym materiale. Rejected oznacza wadliwy komunikat i nie zawiera oceny jakości.

AMQP żądania: MessageId odpowiada messageId w treści, CorrelationId wskazuje simulationId. AMQP odpowiedzi: MessageId = resultId, CorrelationId = messageId źródła. Symulator sprawdza również powiązanie simulationId i zgodność odpowiedzi z zapisanym źródłem.

Brak poprawnego identyfikatora AMQP, niepoprawne UTF-8, przekroczenie 128 KiB, konflikt treści pod tym samym MessageId i niespójna odpowiedź są kierowane do qraft.dead-letters.v1. Odrzucenie kontraktu z identyfikowalnym źródłem jest zapisanym wynikiem Rejected.

Tabela 3. Interfejsy HTTP używane przez panel

Usługa	Operacje
Qraft API	GET /api/inspections; GET /api/inspections/{messageId}; GET .../source; GET /api/batches; GET /api/summary.
Symulator	GET /api/scenarios; POST /api/runs; GET /api/runs; GET /api/runs/{id}; POST /api/messages/{messageId}/replay.
Obie usługi HTTP	GET /health/live; GET /health/ready; dokument OpenAPI.

6. Ponowienia i odporność

Kolejki qraft.inspections.v1 i simulator.results.v1 są trwałe, wiadomości mają trwały tryb dostarczenia, a publikacja używa publisher confirms. Potwierdzenie brokera poprzedza oznaczenie zadania outbox jako opublikowanego.

Odbiorca Qraft potwierdza odbiór ACK po zakończeniu transakcji zapisu wejścia, wyniku, ocen próbek i zadania odpowiedzi. Błąd bazy nie powinien potwierdzać utraconego zapisu. Błąd publikacji pozostawia zadanie do kolejnej próby z opóźnieniem i informacją o błędzie.

Dostarczenie odbywa się co najmniej raz. Po awarii między publikacją a oznaczeniem outboxa może wystąpić retransmisja. Deduplikacja po MessageId oraz hashu treści zachowuje poprzedni ResultId i nie tworzy kolejnej oceny. Zmieniona treść z tym samym identyfikatorem nie może nadpisać historii.

Odbiorca odpowiedzi w symulatorze sprawdza korelację, zachowuje pierwszy wynik i ignoruje jego poprawne powtórzenia. Trwałe outboxy umożliwiają wznowienie wysyłki po powrocie brokera, ale nie są obietnicą braku opóźnień lub wszystkich możliwych awarii.

Aktualna kolejka wyników obsługuje jeden lokalny symulator. Wiele niezależnych baz symulatora współdzielących tę kolejkę wymagałoby oddzielnego routingu odpowiedzi. Nie należy przedstawiać obecnego rozwiązania jako gotowego systemu wielowęzłowego.

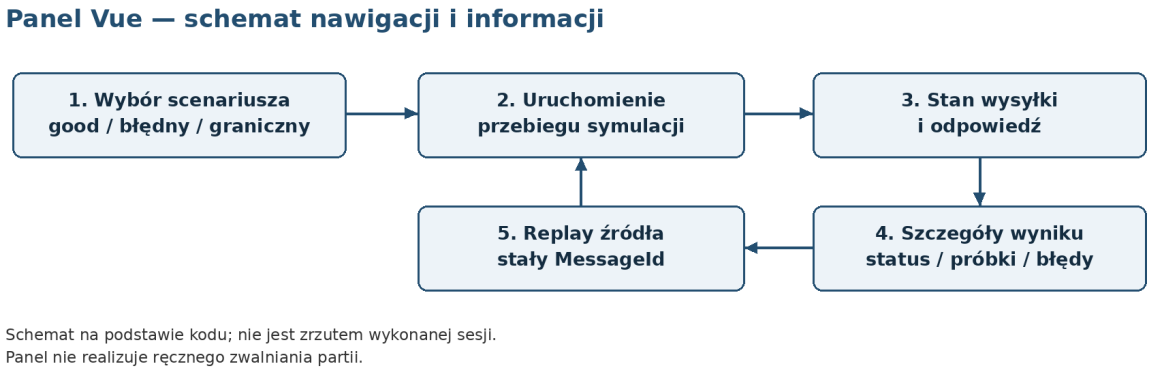
Tabela 4. Oczekiwane zachowanie przy awarii

Przypadek	**Zachowanie**
Broker niedostępny przy wysyłce	Zadanie pozostaje w outboxie; ponowienie po opóźnieniu.
Powtórzenie tego samego wejścia	Ten sam wynik i ResultId, bez dodatkowej oceny.
Ten sam MessageId, inna treść	DLQ; brak nadpisania pierwotnego zapisu.
Błąd zapisu wyniku	Brak ACK przed poprawnym zapisem.
Niepasująca odpowiedź	DLQ zamiast przypisania do obcego źródła.

7. Interfejs i docelowy proces partii

Panel Qraft Lab udostępnia listę scenariuszy, uruchomienie przebiegu, podgląd wysyłki i odpowiedzi oraz szczegóły walidacji i ocen próbek. Operator wybiera scenariusz, wysyła dane, obserwuje stan i otwiera wynik. Funkcja replay ponawia komunikat ze stałym MessageId.

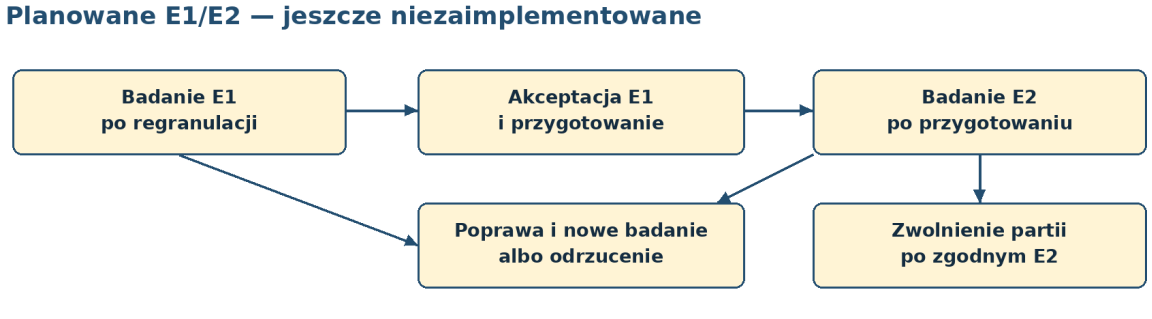
Rysunek 3 jest schematem nawigacji aktualnego panelu, nie zrzutem wykonanej sesji. Nie przedstawia przycisków ręcznego zwalniania ani zatwierdzania partii, których obecny interfejs nie implementuje.



Rysunek 3. Schemat pracy w panelu Vue. Źródło: opracowanie własne.

Docelowo E1 oznacza kontrolę po regranulacji. Po akceptacji E1 pracownik opisze przygotowanie materiału i wykona E2 po ujednorodnieniu oraz przygotowaniu do pakowania. Dopiero pozytywny E2 i zaakceptowany E1 mają pozwolić na zwolnienie partii. Niezgodność ma prowadzić do poprawy i nowego badania albo do odrzucenia.

Rysunek 4 przedstawia wyłącznie planowaną regułę procesu. Implementacja nie wymusza tych przejść. Historia decyzji, autorzy czynności, poprawa i raport całego cyklu wymagają rozszerzenia modelu domenowego.



Rysunek 4. Planowany przebieg E1/E2 — jeszcze niezaimplementowany. Źródło: opracowanie własne.

8. Uruchomienie i weryfikacja

Wymagane wersje narzędzi: .NET SDK 10.0.301, Node.js 22.22.3 i działający Docker z Compose. Polecenie npm run setup odtwarza przypięte zależności i buduje rozwiązanie. npm run dev uruchamia broker, API, worker, symulator oraz Vue; npm run dev:stable pracuje bez dotnet watch.

Panel: http://127.0.0.1:5173. Qraft API: http://127.0.0.1:5280, symulator: http://127.0.0.1:5281. RabbitMQ używa lokalnych portów 5677 i 15677. Ustawienia demonstracyjne znajdują się w .env.example; własnego .env nie zapisuje się w repozytorium.

npm test obejmuje testy C#, testy kontraktów Vue, sprawdzanie typów i budowę frontendu. npm run test:smoke wymaga działającego środowiska i sprawdza pięć scenariuszy oraz retransmisję. npm run test:resilience zatrzymuje broker tego projektu, sprawdza oczekujące zadanie i jego realizację po powrocie brokera.

Aktualne sprawdzenie: kompilacja całego rozwiązania zakończyła się bez ostrzeżeń i błędów; testy C# — 40/40, Vue — 10/10; sprawdzanie typów i budowa frontendu również powiodły się. Rzeczywisty obieg pięciu scenariuszy oraz replay zakończyły się powodzeniem: identyfikatory i wyniki zgadzały się w obu bazach, a retransmisja zachowała ResultId i liczbę ocen. Test przerwy brokera potwierdził oczekujący outbox oraz zapis odpowiedzi po powrocie usługi.

Testy transportu wykonano z natywnym RabbitMQ 4.2.0 i Erlang 27.3.4.6. Kontrolę zatrzymania i powrotu brokera zapewnił izolowany adapter testowy; nie zastępował kolejki ani odpowiedzi. Docker nie był dostępny: npm run setup zakończyło się na sprawdzeniu tego wymagania, natomiast przywrócenie narzędzi i zależności oraz build wykonano oddzielnie. Uruchomienia wariantu Docker nie potwierdzono. Szablon GitHub Actions pozostaje nieaktywny; brak wyniku CI. Szczegółowe wyniki i ograniczenia zawiera WERYFIKACJA.md.

Zatrzymanie: Ctrl+C lub npm run stop. Należy zatrzymać wyłącznie procesy i broker Qraft, zachowując bazy oraz wolumen RabbitMQ. Testy nie wymagają wdrożenia aplikacji do publicznego Internetu.

Tabela 5. Kryteria sprawdzenia aktualnej kopii

Sprawdzenie	**Wymagana obserwacja**
Build i testy C#/Vue	Rzeczywiste zakończenie na przypiętych wersjach; wynik podany w raporcie weryfikacji repozytorium.
Pięć scenariuszy	Zgodne identyfikatory i oczekiwany status w obu bazach.
Replay	Niezmienny ResultId i liczba ocen.
Awaria brokera	Zachowany outbox i odpowiedź po powrocie brokera.

Materiały techniczne: dokumentacja ASP.NET Core i EF Core SQLite (learn.microsoft.com), RabbitMQ .NET Client i Consumer Acknowledgements and Publisher Confirms (rabbitmq.com), Vue 3 (vuejs.org). Źródła objaśniają użyte technologie; limity jakości pochodzą z założeń dydaktycznych projektu.